

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS

Escola Politécnica e de Artes | Design de Interfaces Web

ATIVIDADE PRÁTICA

Depuração de Dashboard

CSS Grid · Flexbox · Botões · Formulários · Alertas

CSS · HTML · JavaScript

Acadêmico(a):		Data:	___ / ___ / ____
Disciplina:	Design de Interfaces Web	Valor:	10,0 pontos

⚠️ ORIENTAÇÕES GERAIS

- ▶ A atividade é de caráter individual. Não é permitido o compartilhamento de trechos de código corrigido entre estudantes.
- ▶ O arquivo layout_erros.html deve ser aberto no navegador (Chrome ou Firefox) antes de iniciar a análise do código-fonte. Redimensionar a janela ajuda a observar os problemas de responsividade.
- ▶ As correções devem ser implementadas diretamente no arquivo recebido. Entregar com o nome layout_corrigido.html.
- ▶ Cada bloco alterado deve ser precedido de um comentário no formato /* CORREÇÃO #N — Conceito */.
- ▶ Não é permitido o uso de frameworks CSS (Bootstrap, Tailwind etc.). Todas as correções devem ser realizadas em CSS puro e JavaScript puro.
- ▶ É permitida a consulta à tabela de conceitos desta atividade e ao material de aula. O uso de ferramentas de contraste e DevTools do navegador é encorajado.

1. Contextualização

Disponibiliza-se o arquivo `layout_erro.html`, que implementa um dashboard fictício de gerenciamento de projetos — TaskFlow. A aplicação abre corretamente no navegador e exibe conteúdo, mas contém dez problemas intencionais distribuídos entre as camadas CSS, HTML e JavaScript, abrangendo layout de página, disposição de componentes e comportamento interativo.

Os erros reproduzem decisões técnicas recorrentes em projetos desenvolvidos sem critérios sistemáticos de layout e sem um sistema de design estabelecido: uso de `float` e `position: absolute` onde CSS Grid seria apropriado, ausência de variantes de componentes, feedback de formulário implementado com recursos inadequados, e ausência completa de responsividade.

Cada problema está sinalizado com um comentário no formato ★ ERRO #N. O comentário delimita a localização, mas não indica o conceito violado nem a forma de correção — ambos são responsabilidade do estudante.

i Sobre o arquivo

- ▶ Utilizar `Ctrl + F` no editor de texto com o termo ★ ERRO para localizar cada marcação.
- ▶ Para observar os problemas de layout, redimensionar a janela do navegador para larguras de 768px e 480px (DevTools → modo responsivo).
- ▶ O erro #1 é estrutural — sua correção impacta diretamente os erros #2, #9 e #10. Recomenda-se resolvê-lo primeiro.
- ▶ Os erros #4 e #5 são complementares: abordam o mesmo componente (botão) em dimensões distintas.

2. Objetivos de Aprendizagem

Ao concluir esta atividade, espera-se que o estudante seja capaz de:

- Reestruturar um layout de página de `position: absolute` e `float` para CSS Grid com `grid-template-areas`, reconhecendo a superioridade semântica e a manutenibilidade da abordagem.
- Substituir um sistema de grade com `float` por CSS Grid com `auto-fill` e `minmax()`, eliminando a dependência de media queries para o ajuste interno de colunas.
- Corrigir uso incorreto de Flexbox (ausência de `align-items`, `flex: 1` e `flex-direction`), verificando o resultado visual no navegador.
- Implementar um sistema de variantes de botão com ao menos quatro perfis semânticos distintos e dois estados interativos ausentes (`:focus-visible` e `:disabled`).
- Substituir `alert()` JavaScript por estados visuais de erro em formulários, utilizando atributos ARIA adequados.
- Criar variantes semânticas de alerta e um componente Toast funcional com animação e remoção automática.
- Implementar responsividade com ao menos dois breakpoints, garantindo legibilidade e usabilidade em viewports de tablet e mobile.

3. Referência Conceitual

A tabela a seguir resume os conceitos avaliados. Utilizá-la como consulta durante a realização dos exercícios.

Conceito	Definição
CSS Grid — grid-template-areas	Propriedade que nomeia regiões do grid. Cada string representa uma linha; células com o mesmo nome se fundem. Permite posicionar elementos em áreas com nomes semânticos (ex.: 'header', 'sidebar', 'main').
CSS Grid — auto-fill + minmax()	Padrão repeat(auto-fill, minmax(min, 1fr)) que cria o máximo de colunas possível, respeitando a largura mínima. Elimina a necessidade de @media query para o grid interno.
Flexbox — align-items	Controla o alinhamento dos itens no eixo transversal. center alinha verticalmente ao meio; stretch (padrão) estica todos à mesma altura; flex-start alinha ao topo.
Flexbox — flex: 1 1 0	Define que um item flex pode crescer (1), encolher (1) e parte de uma base de 0. Usado no item que deve ocupar o espaço restante após os itens fixos. Requer min-width: 0 para evitar overflow.
Flexbox — flex-direction: column	Altera o eixo principal para vertical, empilhando os filhos de cima para baixo. Essencial em componentes como cards, stacks e seções da sidebar.
Botões — variantes	Sistema com ao menos quatro variantes: primário (ação principal), secundário/outline (alternativa), ghost (terciária) e destrutivo (ações irreversíveis). Nunca duas variantes com o mesmo visual para ações de pesos distintos.
Botões — :focus-visible	Pseudo-seletor que aplica o estilo de foco apenas na navegação por teclado (não por mouse). Permite outline visível sem prejudicar a estética em uso comum. Violação de WCAG quando suprimido sem alternativa.
Botões — :disabled	Estado visual que comunica que a ação não está disponível no momento. Requer opacity reduzida e cursor: not-allowed. Não deve ser apenas opacity: 0.
Formulário — estado de erro	Conjunto de elementos: borda colorida no campo, mensagem de erro associada via aria-describedby, e atributo aria-invalid='true'. O alert() JavaScript não constitui feedback visual de formulário.
Alertas — variantes semânticas	Quatro variantes: success (verde), error (vermelho), warning (âmbar), info (azul). Cada uma com ícone, borda lateral colorida e cores de texto/fundo semanticamente distintas. A cor não deve ser o único diferenciador.
Toast	Componente de notificação temporária posicionado com position: fixed fora do fluxo do documento. Requer animação de entrada, duração definida, remoção automática e role ARIA adequado (role='status' ou role='alert').
Responsividade — @media	Blocos @media (max-width: N) que alteram o layout para diferentes viewports. Para um dashboard com sidebar: ao menos um breakpoint para tablet (~1024px) e outro para mobile (~768px).

4. Descrição dos Problemas

Os dez problemas são descritos individualmente a seguir. Para cada um, informam-se a localização no arquivo, a camada técnica, o conceito relacionado, o sintoma observável e uma orientação para correção — sem que a solução seja fornecida de forma direta.

#1

Localização: Seletores `.app-sidebar`, `.app-main` e elementos raiz do layout **Camada:** CSS Grid**Conceito avaliado:** CSS Grid — `grid-template-areas`**Sintoma observável:**

A sidebar ocupa posição absoluta em relação à página, desaparecendo ao rolar o conteúdo. O conteúdo principal usa `margin-left` com valor fixo igual à largura da sidebar — basta a sidebar mudar de tamanho para o layout quebrar. O aside está em `float:right`, saindo do fluxo e causando sobreposições em viewports estreitas.

Orientação para correção:

Identificar o elemento que envolve topbar, sidebar, conteúdo principal e aside e transformá-lo em um contêiner Grid. Nomear as áreas com `grid-template-areas`, especificar as colunas e linhas. Remover `position:absolute`, `margin-left` e `float` dos elementos filhos. Verificar que a topbar, a sidebar e o conteúdo se posicionam corretamente sem valores manuais.

#2

Localização: Seletor `.metrics-row` e elementos `.metric-card` **Camada:** CSS Grid**Conceito avaliado:** CSS Grid — `auto-fill` + `minmax()`**Sintoma observável:**

Os cards de KPI usam `float:left` com largura fixa em pixels. Em viewports menores que a soma das larguras, os cards se empilham de forma imprevisível ou transbordam o container. O `overflow:hidden` no container é necessário apenas como `clearfix` — evidência de que o layout não é nativo do fluxo.

Orientação para correção:

Transformar `.metrics-row` em um container Grid com `repeat(auto-fill, minmax(...))` e `gap` adequado. Remover `float`, `width` fixo e `overflow:hidden`. Definir um valor mínimo de coluna que faça sentido para o tamanho dos cards. Testar redimensionando a janela: o número de colunas deve se ajustar automaticamente.

#3

Localização: Seletor `.content-header` e `.content-header__title` **Camada:** Flexbox**Conceito avaliado:** Flexbox — `align-items` e `flex: 1`**Sintoma observável:**

O título e os botões de ação não estão alinhados verticalmente — itens de alturas diferentes ficam desalinhados no eixo transversal. Os botões não aparecem à direita do título como esperado: o container inteiro está deslocado pela propriedade mal aplicada, em vez de os botões serem empurrados pelo item que cresce.

Orientação para correção:

Verificar qual propriedade controla o alinhamento vertical dos itens filhos em um contêiner Flexbox e adicioná-la ao `.content-header`. Identificar qual item filho deve crescer para ocupar o espaço disponível e empurrar os botões para a extremidade oposta. Remover a propriedade que desloca o contêiner inteiro em vez de um item específico.

#4

Localização: Seletor `.btn` e botões Exportar e Cancelar no HTML **Camada:** CSS — Botões**Conceito avaliado:** Botões — variantes semânticas**Sintoma observável:**

Todos os botões da interface — inclusive Exportar (ação de menor prioridade) e Cancelar (ação de abandono) — têm exatamente o mesmo visual do botão Salvar (ação primária).

O usuário não consegue distinguir visualmente a importância relativa das ações disponíveis.

Orientação para correção:

Criar ao menos três variantes adicionais de botão além do .btn primário existente. Considerar as ações presentes na interface e atribuir a variante adequada a cada botão de acordo com sua importância hierárquica. A variante destrutiva não é necessária neste arquivo, mas as variantes secundária e ghost são.

#5

Localização: Ausência de regras :focus-visible e :disabled no seletor .btn **Camada:** CSS — Botões

Conceito avaliado: Botões — :focus-visible e :disabled

Sintoma observável:

Ao navegar pela interface usando apenas o teclado (Tab), nenhum indicador visual de foco aparece nos botões — a posição atual no fluxo de tabulação é completamente invisível. Não há estilo diferenciado para botões no estado desabilitado, que teriam o mesmo aspecto que botões ativos.

Orientação para correção:

Pesquisar a diferença entre :focus e :focus-visible e qual dos dois preserva a experiência de usuários de mouse sem prejudicar a de usuários de teclado. Adicionar a regra correspondente com outline visível e offset. Criar também a regra para o estado desabilitado com opacidade reduzida e cursor adequado. Testar navegando com Tab no navegador.

#6

Localização: Função validarForm() no JavaScript · Campos #proj-nome e #proj-resp **Camada:** CSS — Formulário

Conceito avaliado: Formulário — estado visual de erro

Sintoma observável:

Ao tentar salvar um projeto com campos obrigatórios vazios, uma caixa de diálogo alert() do navegador bloqueia a interface e exibe uma mensagem genérica. O campo com problema não recebe nenhum indicador visual — borda, ícone ou mensagem de erro posicionada abaixo do campo. Após fechar o alert, o usuário precisa localizar visualmente qual campo está incorreto.

Orientação para correção:

Substituir os alert() por uma abordagem de feedback inline. Para isso, criar no CSS as regras de estado de erro para o campo e para o elemento de mensagem. No JavaScript, adicionar ou remover uma classe de estado no elemento pai do campo afetado, inserir ou atualizar o texto da mensagem de erro, e configurar os atributos de acessibilidade necessários para que o feedback seja anunciado por leitores de tela.

#7

Localização: Seletor .alert e o elemento de alerta no HTML **Camada:** CSS — Alertas

Conceito avaliado: Alertas — variantes semânticas

Sintoma observável:

O único alerta da interface usa sempre o mesmo visual azul, independentemente do tipo de mensagem. A ausência de variantes semânticas impede que o usuário identifique rapidamente a gravidade da informação. Não há ícone e não há borda lateral colorida que reforce a distinção visual.

Orientação para correção:

Criar as variantes de alerta correspondentes aos quatro tipos semânticos. Cada variante deve ter cor de fundo, cor de texto e borda lateral distintas. Adicionar um elemento de ícone (pode ser caractere Unicode ou texto) antes do conteúdo de cada alerta. Aplicar a variante semanticamente correta ao alerta existente no HTML.

#8

Localização: Função salvarProjeto() no JavaScript · Ausência de .toast no CSS **Camada:** CSS + JavaScript**Conceito avaliado:** Toast — componente de notificação**Sintoma observável:**

Ao salvar um projeto com sucesso, um confirm() do navegador interrompe o fluxo da página e exibe uma mensagem interna de sistema. Não existe nenhum componente Toast no arquivo — nem o elemento HTML, nem o CSS, nem a função JavaScript que o controla.

Orientação para correção:

Implementar o componente Toast em três partes: (a) o CSS com posicionamento fixo, aparência visual e animação de entrada definida com @keyframes; (b) o elemento HTML inserido no documento (pode ser criado dinamicamente pelo JS); (c) a função JavaScript que exibe o toast, aguarda um intervalo e o remove. Substituir o confirm() na função salvarProjeto() pela chamada à nova função.

#9

Localização: Seletor .aside-section e seus elementos filhos **Camada:** Flexbox**Conceito avaliado:** Flexbox — flex-direction: column e gap**Sintoma observável:**

As seções do aside têm display:flex mas sem flex-direction:column, o que faz com que o título e os itens de atividade fiquem lado a lado horizontalmente em vez de empilhados. O resultado visual é um colapso do painel lateral, com todo o conteúdo amontoado em uma linha.

Orientação para correção:

Identificar qual propriedade do Flexbox define a direção de empilhamento dos itens filhos e adicioná-la ao .aside-section. Verificar também se há gap ou margin adequados entre os itens para manter o ritmo visual do painel. Testar que o título da seção aparece acima dos itens de lista.

#10

Localização: Ausência de blocos @media em todo o arquivo CSS **Camada:** CSS**Conceito avaliado:** Responsividade — @media queries**Sintoma observável:**

Redimensionando a janela do navegador abaixo de 900px, o conteúdo principal fica parcialmente oculto pela sidebar. Abaixo de 700px, o layout colapsa completamente: sidebar sobrepõe o conteúdo, cards de métrica transbordam a tela, aside invade a área de leitura.

Orientação para correção:

Implementar ao menos dois blocos @media com breakpoints adequados ao tipo de interface (dashboard com sidebar). No breakpoint de tablet, adaptar a largura da sidebar ou ocultá-la; no breakpoint de mobile, reorganizar o layout para coluna única. Se o erro #1 foi corrigido com Grid, a solução consiste em redefinir grid-template-areas e grid-template-columns nos blocos de media.

5. Procedimento de Trabalho

Recomenda-se seguir a sequência abaixo para cada problema:

1. Abrir o arquivo layout_erro.html no navegador e observar o sintoma descrito, incluindo o comportamento em diferentes larguras de janela.
2. Localizar no código-fonte o trecho marcado com ★ ERRO #N.
3. Identificar o conceito violado e registrá-lo na tabela de respostas.
4. Consultar a tabela de conceitos (Seção 3) antes de planejar a correção.
5. Implementar a correção no arquivo, precedendo o trecho com o comentário identificador.
6. Recarregar a página (F5) e verificar o resultado no navegador em ao menos duas larguras de janela.
7. Registrar as alterações e o resultado na tabela de respostas.

Ordem recomendada de resolução

- ▶ Iniciar pelo erro #1 (CSS Grid — layout principal): sua correção cria a estrutura que os erros #2, #9 e #10 dependem.
- ▶ Resolver #2 (auto-fill) logo após, pois ele também é de Grid e está na mesma área do documento.
- ▶ Os erros #4 e #5 são complementares — resolver juntos em sequência.
- ▶ O erro #8 (Toast) é o mais complexo em JavaScript: reservar tempo suficiente e implementá-lo antes das questões de reflexão.
- ▶ O erro #10 (responsividade) deve ser o último — ele confirma que os erros anteriores foram resolvidos corretamente.

6. Registro de Respostas

Preencher a tabela à medida que cada problema for identificado e corrigido.

Erro	Conceito Violado	Alterações Realizadas no Código	Resultado Verificado no Navegador
#1			
#2			
#3			
#4			
#5			
#6			
#7			
#8			
#9			
#10			

6.1 Reflexão Individual

Responder com rigor técnico e linguagem formal, em até quatro linhas cada.

8. O uso de float e position:absolute para layout de página foi amplamente praticado antes do CSS Grid. Quais são as principais limitações dessa abordagem em comparação com Grid? Citar ao menos dois aspectos com base na experiência desta atividade.
9. Qual a diferença prática entre grid-template-areas e a abordagem de colunas numéricas (grid-template-columns com posicionamento por número de linha) para um layout de dashboard? Em que cenário cada abordagem é preferível?
10. O componente Toast implementado no erro #8 exige cuidado com temporização e acessibilidade. Quais atributos ARIA garantem que o toast seja anunciado por leitores de tela, e qual é a diferença semântica entre role='alert' e role='status' nesse contexto?

7. Checklist de Entrega

Verificar todos os itens antes de submeter.

Arquivo e Organização

- ☐ O arquivo entregue tem o nome `layout_corrigido.html`.
- ☐ O arquivo abre sem erros no console do navegador (F12 → Console).
- ☐ Cada bloco corrigido possui comentário identificador `/* CORREÇÃO #N — Conceito */`.
- ☐ Nenhum framework CSS foi adicionado ao arquivo.

Layout (Grid e Flexbox)

- ☐ O layout de página usa CSS Grid com `grid-template-areas` — sem `float` nem `position: absolute` estrutural.
- ☐ Os cards de métrica usam CSS Grid com `auto-fill` e `minmax()` — sem `float` nem largura fixa.
- ☐ O cabeçalho de seção alinha título e botões corretamente com Flexbox.
- ☐ O painel aside empilha seus itens verticalmente com `flex-direction: column`.

Componentes

- ☐ Existem ao menos quatro variantes de botão com aparências visuais distintas.
- ☐ Botões exibem indicador de foco visível ao navegar com Tab.
- ☐ Campos obrigatórios exibem estado de erro visual (borda + mensagem inline) ao falhar na validação.
- ☐ Existem ao menos três variantes semânticas de alerta (`success`, `error`, `warning`).
- ☐ O componente Toast aparece após salvar, anima-se e desaparece automaticamente.

Responsividade

- ☐ Em viewports de tablet (~1024px), o layout se adapta sem sobreposições.
- ☐ Em viewports de mobile (~768px), o conteúdo ocupa a largura total sem sidebar visível.
- ☐ Os cards de métrica se reorganizam automaticamente em viewports estreitas.

8. Critérios de Avaliação

Cada um dos dez problemas vale 1,0 ponto, distribuído igualmente entre identificação e correção funcional.

Erro	Conceito corretamente identificado (0,5 pt)	CSS/JS corrigido e funcional no navegador (0,5 pt)	Observações do professor
#1			
#2			
#3			
#4			
#5			
#6			
#7			
#8			
#9			
#10			

Observações sobre a pontuação

- ▶ Identificação correta (0,5 pt): o conceito foi nomeado adequadamente na tabela de respostas e corresponde ao problema marcado no arquivo.
- ▶ Correção funcionando (0,5 pt): a alteração elimina o sintoma observável no navegador sem introduzir novos problemas. Ausência de comentário identificador implica redução de 0,1 ponto por item.
- ▶ Erro #1: a pontuação máxima exige remoção completa de float e position:absolute estruturais, substituídos por CSS Grid funcional.
- ▶ Erro #8: a pontuação máxima exige as três partes do componente — CSS com animação, criação do elemento e remoção automática via JavaScript.
- ▶ Reflexão individual: avaliada qualitativamente como critério de desempate e indicador de compreensão conceitual.

Item de Avaliação	Pontuação Máxima	Nota Obtida
Erros #1 a #10 — Identificação do conceito violado (0,5 pt cada)	5,0 pts	
Erros #1 a #10 — Correção implementada e funcional no navegador (0,5 pt cada)	5,0 pts	
TOTAL	10,0 pts	

9. Gabarito

Uso exclusivo do professor — não distribuir antes da entrega da atividade.

Erro	Conceito	Correção Esperada
#1	CSS Grid — grid-template-areas	Substituir position:absolute na sidebar e margin-left manual no .app-main por um layout com display:grid no elemento raiz. Definir grid-template-areas com as regiões 'topbar', 'sidebar', 'main'. Especificar grid-template-columns (ex.: 220px 1fr) e grid-template-rows (56px 1fr). Remover position:absolute, margin-left e margin-top manuais.
#2	CSS Grid — auto-fill + minmax()	Substituir float:left e width:180px nos .metric-card por um container .metrics-row com display:grid e grid-template-columns: repeat(auto-fill, minmax(180px, 1fr)). Adicionar gap adequado. Remover float, overflow:hidden e width fixo.
#3	Flexbox — align-items e flex: 1	Adicionar align-items: center ao .content-header para alinhar verticalmente título e botões. Adicionar flex: 1 ao .content-header__title para que ele cresça e empurre os botões para a direita. Remover margin-left: auto do container.
#4	Botões — variantes	Criar ao menos três variantes adicionais: .btn-secondary (fundo transparente, borda e texto na cor primária), .btn-ghost (fundo transparente, texto em cinza, borda neutra) e .btn-danger (fundo vermelho). Aplicar .btn-ghost ou .btn-secondary ao botão Exportar e ao botão Cancelar.
#5	Botões — :focus-visible e :disabled	Adicionar regra .btn:focus-visible com outline visível (ex.: outline: 3px solid + outline-offset). Adicionar regra .btn:disabled com opacity reduzida e cursor: not-allowed, além de pointer-events: none. Verificar que a navegação por Tab exibe o indicador de foco claramente.
#6	Formulário — estado visual de erro	Criar seletor .form-group--error .form-group input (border-color vermelha, background levemente rosado). Criar .form-error-msg com color, font-size e display corretos. Substituir alert() na função validarForm() por: adicionar .form-group--error ao pai do campo, inserir elemento de mensagem de erro com id referenciado em aria-describedby, e definir aria-invalid='true' no input.
#7	Alertas — variantes semânticas	Criar as classes .alert-success, .alert-error, .alert-warning, cada uma com background, border-left colorida e color de texto semanticamente distintos. Adicionar ícone (texto ou SVG) antes do conteúdo. Aplicar a variante semanticamente correta ao alerta de boas-vindas existente.
#8	Toast — componente de notificação	Criar o seletor .toast com position:fixed, bottom e right definidos, z-index alto, border-radius, box-shadow e animação de entrada (@keyframes slide-in com translateX ou translateY + opacity). Implementar função mostrarToast(mensagem) em JavaScript que insere o elemento, o exibe e o remove após 3–4 segundos via setTimeout. Substituir o confirm() em salvarProjeto() pela chamada à função.
#9	Flexbox — flex-direction: column e gap	Adicionar flex-direction: column ao .aside-section para empilhar os filhos verticalmente em vez de horizontalmente. Adicionar gap (ex.: 0 ou valor pequeno) para controlar o espaçamento entre os elementos filhos diretos. Verificar que .aside-section__title e os .aside-item aparecem em ordem vertical no navegador.
#10	Responsividade — @media queries	Implementar ao menos dois blocos @media. Em max-width: 1024px: recolher ou ocultar a sidebar, expandir o conteúdo principal para 100% da largura. Em max-width: 768px: remover o aside do fluxo ou ocultá-lo, ajustar o padding do .app-main, empilhar os cards de métrica em coluna única. Se o erro #1 foi corrigido com

Erro	Conceito	Correção Esperada
		Grid, basta redefinir grid-template-areas e grid-template-columns nos blocos de media.

Bons estudos.